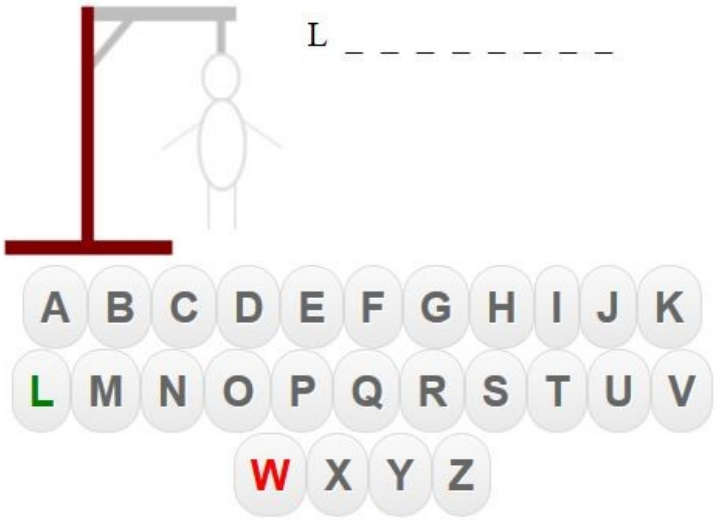
Développement d’un jeu du pendu en ligne



Contexte du projet :

Dans le cadre de la formation en développement web, les étudiants doivent créer un jeu complet et interactif : **le jeu du pendu**, intégrant un backend PHP, une base de données MySQL, une interface dynamique en HTML/CSS, ainsi qu’une gestion d’utilisateurs sécurisée.

L’objectif est de maîtriser l’ensemble de la chaîne de développement :

- Frontend
- Backend
- Base de données
- Sécurisation
- Gestion de sessions
- Travail collaboratif sur Git

Cahier des charges

1. Fonctionnalités principales

1.1. Gestion des utilisateurs

- **Inscription et Connexion :**
  L'utilisateur peut s'inscrire avec un email et un **mot de passe valide (suivant les recommandations de la CNIL)**.
- **Validation des données :**
  - Email conforme
  - Mot de passe valide selon la CNIL
- **Sessions :**
  - Lorsqu'un utilisateur est connecté, il ne peut plus accéder à /login ou /register : il est redirigé automatiquement vers la **page du jeu**.
  - Lorsqu'un utilisateur **n'est pas connecté**, il ne peut accéder à **aucune page du jeu** : redirection automatique vers /login.
- **Déconnexion :**
  Un bouton permet de fermer correctement la session utilisateur.

1.2. Jeu du pendu

Depuis le menu principal, l'utilisateur peut :

- **Commencer une nouvelle partie**
- **Voir le classement**
- **Accéder à la page "Règles du jeu"**

Choix du niveau

Lors d'une nouvelle partie, l'utilisateur choisit un niveau :

- **Facile**
- **Moyen**
- **Difficile**

Chaque niveau utilise une liste de mots différente, stockée en base de données.

1.3. Liste des mots et tirage

La base contient au minimum une table regroupant :

- le mot
- sa difficulté (easy / medium / hard)

Un mot est tiré aléatoirement selon la difficulté.

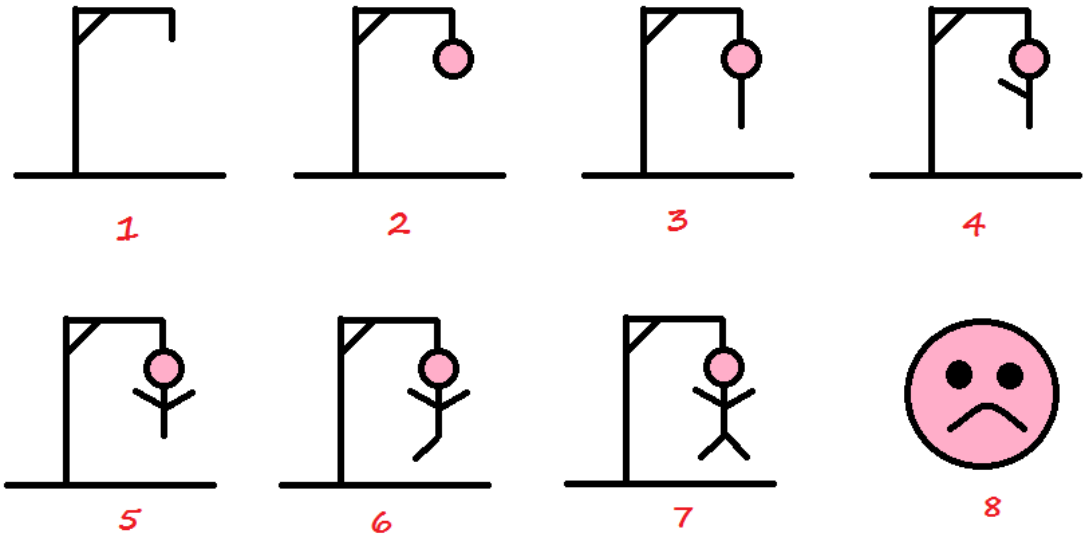
Pour aider les joueurs, certaines lettre devront etre afficher dès la début de la partie suivant le niveau de difficulté :

- **Facile :** 2 lettres aléatoires révélées
- **Moyen :** 1 lettre révélée
- **Difficile :** aucune lettre révélée

Lorsqu'une lettre révélée apparaît plusieurs fois, **toutes ses occurrences doivent apparaître**.

Exemple :
Mot : **CROCODILE**
Lettre tirée : **O**
Affichage initial :
_ _ O _ O _ _ _ _

1.4. Déroulement de la partie



À chaque tour, l'utilisateur peut :

1. **Proposer une lettre**, ou
2. **Proposer un mot entier**

Règles :

- **Lettre correcte :**
  Toutes les occurrences sont révélées dans le mot.
- **Lettre incorrecte :**
  Le joueur perd **une vie**.
- **Lettre déjà tentée auparavant :**
  Ne fait pas perdre de vie.
- **Mot correct :**
  Victoire immédiate + bonus.
- **Mot incorrect :**
  Perte de **2 vies**.

L'interface doit afficher :

- Le mot avec les lettres trouvées dans leur ordre exact
- Les **emplacements vides** restants (ex : _ R O _ O _ I _ E)
- Les lettres déjà testées
- Le nombre de vies restantes
- Le visuel du pendu (ou une barre de vie selon la direction artistique choisie)
-

1.5. Fin de partie & scoring

Les étudiants sont **libres** dans la manière de calculer le score final.
Ils doivent :

- Prendre en compte la difficulté
- Le nombre d'essais
- Bonus/malus selon victoire ou défaite
- Décrire ces règles dans une **page dédiée « Règles du jeu »**

À la fin de chaque partie :

- Si le joueur gagne → il gagne des points
- S'il perd → il perd des points

Les résultats sont enregistrés en base.

1.6. Classement

- Affiche les meilleurs joueurs en fonction du total de points.
- Le classement est **calculé dynamiquement**, pas stocké dans une table fixe.
- Les règles de points et de bonus suivant la difficulté sont libre. Une page avec les régles du jeu devra etre présente sur le site

2. Technologies utilisées

Frontend

- HTML5 / CSS3
- Interface responsive
- Affichage du jeu (mot, lettres, vies, pendu ou barre d’énergie)

Backend

- PHP pour :
    - Formulaires (inscription, connexion)
    - Gestion des sessions
    - Logique du jeu
    - Gestion des erreurs
    - Classement
- Sécurisation :
    - Protection contre les injections SQL
    - Protection XSS
    - Validation des entrées
    - Utilisation de requêtes préparées
    - Hashage des mots de passe

3. Base de données (MySQL)

Minimum obligatoire.
Les étudiants peuvent ajouter d’autres tables si nécessaire.

Table users

- id
- email
- password
- created_at

Table words

- id
- word
- difficulty (easy / medium / hard)

Table games

- id
- user_id
- word_id
- difficulty
- attempts
- victory (0/1)
- points
- created_at

Travail collaboratif (obligatoire)

Le projet est à réalisé par groupe de 3

- Tout le projet doit être réalisé sur Git
- Un README propre et complet est obligatoire. Il doit contenir la documentation techique du projet.
- **Chaque membre doit travailler sur l’intégralité du projet et toute personne peut être interrogée sur n’importe quelle partie du code**
- Commits réguliers et clairs avec messages explicites et cohérents

- **Participation égale :** Toute inégalité flagrante pourra être pénalisée

Critères d’évaluation

Critères d’évaluation	/20
Qualité visuelle : Esthétique générale, cohérence des couleurs, alignement, lisibilité.	/ 1
Ergonomie : Simplicité d’utilisation, navigation fluide, boutons d’action clairs.	/2
Responsive design : Capacité à s’adapter à différents formats d’écran (PC, tablette, smartphone).	/1
Gestion des utilisateurs : Inscription, connexion, validation des données, hachage de mots de passe.	/2
Logique et déroulement du jeu : Gagner et perdre une partie, calcul des points.	/5
Sauvegarde des résultats : Enregistrement des résultats dans la base de données et affichage du classement	/2
Validation des formulaires : Validation correcte des emails, mots de passe, protection contre les soumissions incorrectes et les injections SQL.	/1
Travail en équipe, utilisation de git, pas de code duppliqué	/2
Sécurisation du projet	/2
Documentation technique du projet	/2